



# Incremental Lossless Graph Summarization

Jihoon Ko\*

KAIST AI

jihoonko@kaist.ac.kr

Yunbum Kook\*

Dept. of Mathematical Science, KAIST

yb.kook@kaist.ac.kr

Kijung Shin<sup>†</sup>

KAIST AI & EE

kijungs@kaist.ac.kr

## ABSTRACT

Given a fully dynamic graph, represented as a stream of edge insertions and deletions, how can we obtain and incrementally update a lossless summary of its current snapshot?

As large-scale graphs are prevalent, concisely representing them is inevitable for efficient storage and analysis. Lossless graph summarization is an effective graph-compression technique with many desirable properties. It aims to compactly represent the input graph as (a) a *summary graph* consisting of supernodes (i.e., sets of nodes) and superedges (i.e., edges between supernodes), which provide a rough description, and (b) *edge corrections* which fix errors induced by the rough description. While a number of batch algorithms, suited for static graphs, have been developed for rapid and compact graph summarization, they are highly inefficient in terms of time and space for dynamic graphs, which are common in practice.

In this work, we propose MoSSo, the first incremental algorithm for lossless summarization of fully dynamic graphs. In response to each change in the input graph, MoSSo updates the output representation by repeatedly moving nodes among supernodes. MoSSo decides nodes to be moved and their destinations carefully but rapidly based on several novel ideas. Through extensive experiments on 10 real graphs, we show MoSSo is (a) **Fast and ‘any time’**: processing each change in near-constant time (less than 0.1 millisecond), up to **7 orders of magnitude faster** than running state-of-the-art batch methods, (b) **Scalable**: summarizing graphs with hundreds of millions of edges, requiring sub-linear memory during the process, and (c) **Effective**: achieving comparable compression ratios even to state-of-the-art batch methods.

## ACM Reference Format:

Jihoon Ko, Yunbum Kook, and Kijung Shin. 2020. Incremental Lossless Graph Summarization. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '20)*, August 23–27, 2020, Virtual Event, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3394486.3403074>

## 1 INTRODUCTION

How can we extract a lossless summary from a dynamic graph and update the summary, reflecting changes in forms of edge additions and deletions? Can we perform each update in near-constant time while maintaining a concise summary?

\*Equal Contribution. <sup>†</sup>Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

KDD '20, August 23–27, 2020, Virtual Event, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7998-4/20/08...\$15.00

<https://doi.org/10.1145/3394486.3403074>

**Table 1: Comparison of lossless graph summarization methods. MoSSo is fast, space-efficient, and online.**

|                                   | [21] | [13] | [27] | MoSSo (Proposed) |
|-----------------------------------|------|------|------|------------------|
| Takes near-linear time            | ×    | ✓    | ✓    | ✓                |
| Requires sub-linear space         | ×    | ×    | ×    | ✓                |
| Handles inserted edges (or nodes) | ×    | ×    | ×    | ✓                |
| Handles deleted edges (or nodes)  | ×    | ×    | ×    | ✓                |

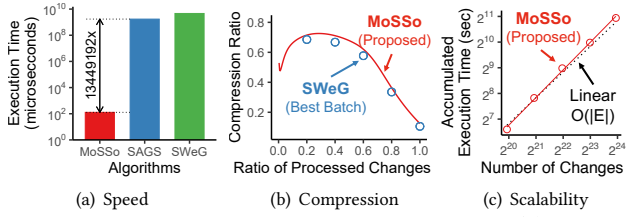
Datasets representing relationships between objects are universal in both academia and industry, and graphs are widely used as a simple but powerful representation of such data. For example, graphs naturally represent social networks, WWW, internet topologies, citation networks, and even protein-protein interactions.

With the emergence of big data, real-world graphs have increased dramatically in size, and many of them are still evolving over time. For example, the number of active users in Facebook (i.e., the number of active nodes in the Facebook graph) has increased dramatically from 240 millions to 2.4 billions in the last decade. Such large dynamic graphs are naturally represented as a *fully dynamic graph stream*, i.e., a stream of edge insertions and deletions over time.

To manage large-scale graphs efficiently, compactly representing graphs has become important. Moreover, compact representations allow a larger portion to be stored in main memory or cache and thus can speed up computation on graphs [6, 28]. Thus, many graph-compression techniques have been proposed: relabeling nodes [4, 7, 8], pattern mining [6], lossless graph summarization [13, 21, 27], lossy graph summarization [2, 16, 24], to name a few.

Lossless graph summarization is one of the most effective graph compression techniques. Note that we use the term “lossless graph summarization” to indicate this specific compression technique throughout this paper. Applying lossless graph summarization to a graph  $G = (V, E)$  yields (a) a summary graph  $G^* = (S, P)$ , where  $S$  is a partition of  $V$  (i.e., each element of  $S$  is a subset of  $V$ , and every node in  $V$  belongs to exactly one element of  $S$ ) and  $P$  is a set of pairs of two elements in  $S$ , and (b) edge corrections  $C = (C^+, C^-)$ . Let  $\hat{G}$  be the graph where all pairs of nodes in each pair of subsets in  $P$  are connected and the others are not. Then,  $C^+$  and  $C^-$  are the sets of edges to be added to and removed from  $\hat{G}$ , respectively, for recovering the original graph  $G$ . In addition to yielding compact representations, lossless graph summarization stands out among many compression techniques due to the following desirable properties:

- **Queryable**: neighborhood queries (i.e., retrieving the neighborhood of a query node) can be answered rapidly from a summary graph and edge corrections (Lemma 1 in Sect. 3.6). Neighborhood queries are the key building blocks repeatedly called in numerous graph algorithms (DFS, Dijkstra’s algorithm, PageRank, etc).
- **Combinable**: summary graphs and edges corrections, which are in the form of graphs, can be further compressed by any graph-compression methods. Combined with [4, 6–8], lossless graph summarization achieves up to 3.4× additional compression [27].



**Figure 1: MoSSo is fast, effective, and scalable.** (a) **Fast:** processes a change up to 7 orders of magnitude faster than the fastest batch method. (b) **Effective:** gives comparable compression rates to the best batch method. (c) **Scalable:** handles each change in near-constant time, i.e., its total runtime is near-linear in the number of changes. See Sect. 4 for details.

The lossless graph summarization problem [21], whose goal is to find the most concise representation in the form of a summary graph and edge corrections, has been formulated only for static graphs. Thus, as shown in Table 1, all existing solutions [13, 21, 27] are prohibitively inefficient for dynamic graphs, represented as fully dynamic graph streams. Specifically, since these batch algorithms are not designed to allow for incremental changes in the input graph, they should be rerun from scratch to reflect such changes.

In this work, we formulate a new problem, the goal of which is lossless summarization of a fully dynamic graph. Then, we propose the first incremental algorithm MoSSo (**M**ove if **S**aved, **S**tay **o**therwise) for the problem. In response to each change in the input graph, MoSSo updates the summary graph and edge corrections by moving nodes among supernodes. (a) **CORRECTIVE ESCAPE:** injects randomness in order to escape a local optimum and to cope with changing optima, (b) **FAST RANDOM:** performs neighborhood sampling, which is repeatedly executed in MoSSo, in near-constant time on the output representation without having to retrieve all neighbors,<sup>1</sup> and (c) **CAREFUL SELECTION:** when selecting a new supernode into which a node moves, selects it effectively using coarse clustering. Note that the coarse clustering is distinct from graph summarization, which we refer to as fine clustering.

We evaluate MoSSo with respect to speed and compression rate on 10 real-world graphs with up to 0.3 billion edges. Specifically, we compare MoSSo with state-of-the-art batch algorithms for lossless graph summarization as well as streaming baselines based on greedy and Markov chain Monte Carlo methods. Through theoretical and empirical analyses, we show the following merits of MoSSo:

- **Fast and ‘any time’:** It takes near-constant time to process each change (0.1 millisecond per change), and is up to 10 million times faster than state-of-the-art batch algorithms (Figs. 1(a), 1(c)).
- **Scalable:** It requires sub-linear memory (Thm. 4) while summarizing graph streams with up to hundreds of millions of edges.
- **Effective:** It does not pale in comparison with state-of-the-art batch algorithms in terms of compression rates (Fig. 1(b)).

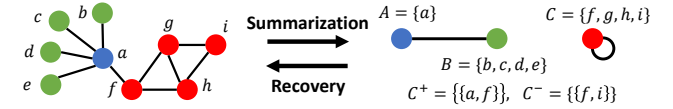
**Reproducibility:** The code and datasets used in the paper are available at <http://dmlab.kaist.ac.kr/mosso/>.

In Sect. 2, we introduce some notations and concept, and we formally formulate a new problem whose goal is to incrementally summarize graph streams. In Sect. 3, we present MoSSo and analyze it theoretically. In Sect. 4, we provide experimental results. After reviewing previous work in Sect. 5, we offer conclusions in Sect. 6.

<sup>1</sup>Naively, we retrieve all neighbors of a query node from the output representation, which takes time proportional to its degree, and then uniformly sample some.

**Table 2: Table of Symbols.**

| Symbol                                             | Meaning                                                  |
|----------------------------------------------------|----------------------------------------------------------|
| Symbols for the problem definition (Sect. 2.1)     |                                                          |
| $G=(V, E)$                                         | a simple undirected graph with nodes $V$ and edges $E$   |
| $N(u)$                                             | neighborhood of node $u$                                 |
| $\deg(u)$                                          | degree of node $u$                                       |
| $\{e_t\}_{t=0}^{\infty}$                           | a fully dynamic graph stream                             |
| $e_t=\{u, v\}^+$                                   | edge insertion at time $t$                               |
| $e_t=\{u, v\}^-$                                   | edge deletion at time $t$                                |
| $G_t$                                              | $G$ at time $t$                                          |
| $G^*=(S, P)$                                       | a summary graph with supernodes $S$ and superedges $P$   |
| $C^+$                                              | set of edges to be inserted                              |
| $C^-$                                              | set of edges to be deleted                               |
| $\varphi$                                          | objective function that we aim to minimize               |
| Symbols for the algorithm descriptions (Sect. 3.1) |                                                          |
| $y$                                                | a testing node                                           |
| $TP(u)$                                            | testing pool related to node $u$                         |
| $TN(u)$                                            | testing nodes related to node $u$                        |
| $z$                                                | a candidate                                              |
| $CP(y)$                                            | candidate pool for testing node $y$                      |
| $E_{AB}$                                           | edges between supernodes $A$ and $B$                     |
| $T_{AB}$                                           | all possible edges between supernodes $A$ and $B$        |
| $S_v$                                              | supernode in $G^*$ that contains node $v$                |
| $N(S_v)$                                           | neighborhood of supernode $S_v$ in a summary graph $G^*$ |
| $C^+(u)$                                           | edges in $C^+$ incident to node $u$                      |
| $C^-(u)$                                           | edges in $C^-$ incident to node $u$                      |



**Figure 2: Example of lossless graph summarization.** The input graph has 10 edges, while a summary graph and edge corrections have only 4 (super) edges, i.e.,  $|P|+|C^+|+|C^-|=4$ . The optimal encoding in Sect. 3.1 creates the superedge  $\{A, B\}$  since  $|E_{AB}| > \frac{|T_{AB}|+1}{2}$  but not  $\{A, C\}$  since  $|E_{AC}| \leq \frac{|T_{AC}|+1}{2}$ .

## 2 NOTATIONS AND PROBLEM SETUP

We first introduce some important notations and concepts, which are listed and illustrated in Table 2 and Fig. 2. Then, we formulate a new problem, namely incremental lossless graph summarization.

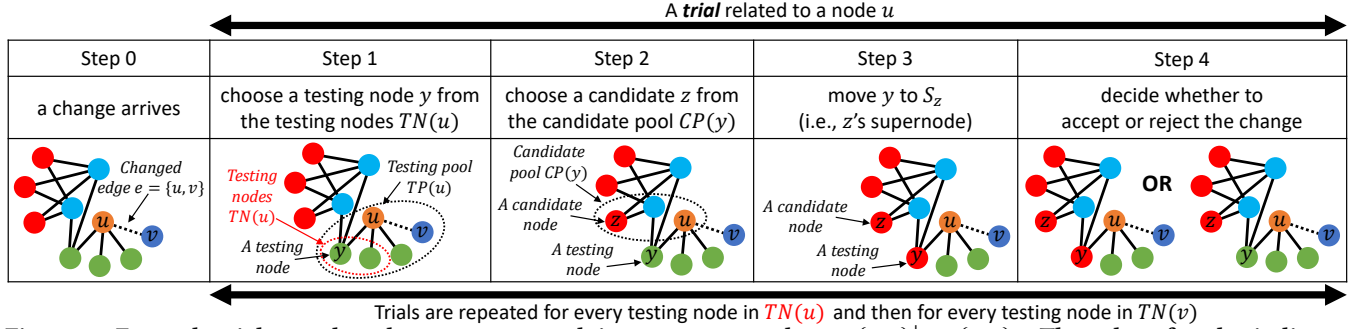
### 2.1 Notations and Concepts

**Graph:** A graph  $G=(V, E)$  consists of a set  $V$  of nodes and a set  $E$  of edges. We especially consider a simple undirected graph; each edge  $\{u, v\} \in E$  is an unordered pair of distinct nodes  $u, v \in V$ . We denote the neighborhood of a node  $u$  (i.e., the set of nodes adjacent to  $u$ ) as  $N(u) \subset V$ , and we define the degree of  $u$  as  $|N(u)|$ .

**Summary Graph:** A graph  $G^*=(S, P)$  is a summary graph of a graph  $G=(V, E)$  if its node set  $S$  is a partition of  $V$ , i.e., if  $V=\bigcup_{\alpha} S_{\alpha}$  with  $S_{\alpha} \cap S_{\beta}=\emptyset$  for any distinct  $S_{\alpha}, S_{\beta} \in S$ . To avoid confusion, we call nodes and edges in  $G$  simply nodes and edges, while we call them in its summary graph  $G^*$  supernodes and superedges. We denote the supernode containing a node  $u$  as  $S_u$  and the superedge between two supernodes  $S_u, S_v \in S$  as  $\{S_u, S_v\} \in P$ .

**Edge Corrections & Recovery:** Given a summary graph  $G^*=(S, P)$  of a graph  $G=(V, E)$ , we obtain a graph  $\hat{G}=(V, \hat{E})$  by connecting every pair of nodes between two neighboring supernodes, i.e.,  $\{u, v\} \in \hat{E}$  if and only if  $u \neq v$  and  $\{S_u, S_v\} \in P$ . We can say  $G^*$  roughly describes  $G$  if  $\hat{G}$  is similar to  $G$ . Moreover, with edge corrections  $C=(C^+, C^-)$  where  $C^+:=E-\hat{E}$  and  $C^-:=\hat{E}-E$ , the original graph  $G=(V, E)$  is exactly recovered from  $G^*$  as follows:

$$V \leftarrow \bigcup_{S_{\alpha} \in S} S_{\alpha}, \quad E \leftarrow (\hat{E} \cup C^+) \setminus C^-.$$



**Figure 3: Example trial to update the summary graph in response to a change  $\{u, v\}^+$  or  $\{u, v\}^-$ . The color of nodes indicates their membership to supernodes. For an input node  $u$  (and then for  $v$ ), the algorithms presented in Sect. 3 set a testing pool  $TP(u)$  and obtain testing nodes  $TN(u) \subseteq TP(u)$ . Then, they repeat trials (i.e., the above steps 1-4) for every testing node in  $TN(u)$ . Note that the algorithms set  $TP(u)$ ,  $TN(u)$ , and  $CP(y)$  in different ways.**

That is,  $G^*$  and  $C$  losslessly summarize  $G$ . Given  $G$ , the *lossless summarization problem* [21] is to find the most concise  $G^*$  and  $C$ .

**Fully Dynamic Graph Stream:** We define a *fully dynamic graph stream* by a sequence  $\{e_t\}_{t=0}^\infty$  of changes. Each change  $e_t$  is either an *edge addition*  $e_t = \{u, v\}^+$  or an *edge deletion*  $e_t = \{u, v\}^-$  between distinct nodes  $u, v \in V$ . For each edge addition  $e_t = \{u, v\}^+$ ,  $u$  and/or  $v$  may be new nodes unseen until the current time  $t$ . Since node additions and deletions are expressed as a series of the additions and deletions of their adjacent edges, respectively, a sequence of edge additions and deletions is expressive enough to represent a dynamic graph with new and deleted nodes.

An empty graph  $G_0 = \emptyset$  evolves according to  $\{e_t\}_{t=0}^\infty$ . We define the graph at time  $t$  by  $G_t = (V_t, E_t)$ , where  $V_t$  and  $E_t$  are obtained in the inductive way: (1) **addition**: if  $e_{t-1} = \{u, v\}^+$ , then  $E_t = E_{t-1} \cup \{\{u, v\}\}$  and  $V_t = V_{t-1} \cup \{u, v\}$ , (2) **deletion**: if  $e_{t-1} = \{u, v\}^-$ , then  $E_t = E_{t-1} \setminus \{\{u, v\}\}$  and  $V_t = V_{t-1}$ . We assume  $\{u, v\} \notin E_{t-1}$  for every edge addition and  $\{u, v\} \in E_{t-1}$  for every edge deletion. That is, we assume that the graph stream is sound.

## 2.2 Problem Formulation

Given a fully dynamic graph  $G_t$  evolving under  $\{e_t\}_{t=0}^\infty$ , the most concise summary graph  $G_t^*$  and edge corrections  $C_t$  change in response to each change  $e_t$ . Since recomputing concise  $G_t^*$  and  $C_t$  for each  $G_t$  from scratch is prohibitive for large-scale graphs, an incremental update on  $G_t^*$  and  $C_t$  to obtain concise  $G_{t+1}^*$  and  $C_{t+1}$  is highly desirable. Thus, we formulate a new problem, namely *incremental lossless graph summarization*, in Problem 1.

**PROBLEM 1. (Incremental Lossless Graph Summarization).**

- (1) **Given:** a fully dynamic graph stream  $\{e_t\}_{t=0}^\infty$
- (2) **Retain:** a summary graph  $G_t^* = (S_t, P_t)$  and edge corrections  $C = (C_t^+, C_t^-)$  of graph  $G_t$  at current time  $t$
- (3) **To Minimize:** the size of output representation, i.e.,
 
$$\varphi(t) := |P_t| + |C_t^+| + |C_t^-| \quad (1)$$

The objective  $\varphi(t)$  measures the size of the summary graph  $G_t^*$  and the edge corrections  $C_t$ . In  $\varphi(t)$ , the number of supernodes, which is marginal compared to that of (super) edges appearing in output representations, is disregarded for simplicity. This objective was first proposed by [21] under the *Minimum Description Length* principle [25], which originates from information theory. Then, a number of algorithms for lossless summarization of static graphs have been developed based on the objective [13, 21, 27].

In Problem 1, there are almost infinitely many possible summary graphs [3] and moreover, the optimal solution is subject to change at any time. Thus, we focus on streaming algorithms for finding approximate solutions. Note that, even for static graphs, only heuristics without guarantees have been studied.

## 3 PROPOSED METHOD

In this section, we propose MoSSo, a fast and space-efficient streaming algorithm for lossless graph summarization. We also present baseline streaming algorithms and a preliminary version of MoSSo, while overcoming their drawbacks addressed in full-fledged MoSSo.

We first introduce terms and subroutines used in the algorithms. Then, we present the baseline algorithms. After that, we present a preliminary version of MoSSo and then full-fledged MoSSo, with a focus on ideas that leads to improved scalability and compression rates. Lastly, we analyze the time and space complexities of MoSSo.

### 3.1 Common Terms and Subroutines (Fig. 3)

We define some terms and subroutines commonly used in the algorithms presented in the following subsections.

**Common Terms:** Assume that  $\{u, v\}$  is inserted or deleted at current time  $t$  (i.e.,  $e_t = \{u, v\}^+$  or  $\{u, v\}^-$ ). We call nodes  $u$  and  $v$  *input nodes*. A *trial related to  $u$*  on a node  $x$  is the attempt to change the supernode membership  $S_x$  of the node  $x$ , which is somehow related to  $u$ . A *testing pool*  $TP(u)$  is the set of nodes on which a trial related to  $u$  can be proceeded. A *testing node* is a node on which a trial is actually proceeded at current time  $t$ . We let  $y$  be each testing node and  $TN(u) \subseteq TP(u)$  be the set of all testing nodes at current time  $t$ . A *candidate*  $z$  is a node in the supernode into which the testing node  $y$  tries moving. Note that this move can be rejected and reverted. A *candidate pool*  $CP(y) \subseteq V$  is a set of all possible candidates given to the testing node  $y$ .

**Trial:** A subroutine called *trial related to  $u$*  is described as follows:

**A trial related to  $u$  (the terms defined above are used):**

- (1) Select a *testing node*  $y$  from  $TN(u)$ .
- (2) Select a *candidate*  $z$  from  $CP(y)$ .
- (3) Propose moving  $y$  into  $S_z$ .
- (4) Determine whether to accept or reject the move, based on the change in the objective  $\varphi$  (when assuming the optimal encoding described in the next paragraph).

This reflects why we name the final algorithm MoSSo (**M**ove if **S**aved, **S**tay otherwise). This trial is repeated for every testing node

in  $TN(u)$ . The algorithms presented in the following subsections are distinguished by how they (1) set a testing pool  $TP(u)$ , (2) extract testing nodes  $TN(u)$  from  $TP(u)$ , (3) set a candidate pool  $CP(y)$ , (4) propose a candidate from  $CP(y)$ , and (5) accept the proposal.

**Optimal Encoding:** While finding the *optimal* set  $S$  of supernodes, which minimizes the objective  $\varphi$ , is challenging, finding optimal  $P$  and  $C$  for current  $S$  is straightforward. For each supernode pair  $\{A, B\}$ , let  $E_{AB} := \{\{u, v\} \in E \mid u \in A, v \in B, u \neq v\}$  and  $T_{AB} := \{\{u, v\} \subseteq V \mid u \in A, v \in B, u \neq v\}$  be the sets of existing and potential edges between  $A$  and  $B$ , respectively. Then, the edges between  $A$  and  $B$  (i.e.,  $E_{AB}$ ) are optimally encoded as follows:

**Optimal encoding for the edges in  $E_{AB}$ :**

- (1) If  $|E_{AB}| \leq \frac{|T_{AB}|+1}{2}$ , then add all edges in  $E_{AB}$  to  $C^+$ .
- (2) If  $|E_{AB}| > \frac{|T_{AB}|+1}{2}$ , then add the superedge  $\{A, B\}$  to  $P$  and  $T_{AB} \setminus E_{AB}$  to  $C^-$ .

Note that adding all edges in  $E_{AB}$  to  $C^+$  increases  $\varphi$  by  $|E_{AB}|$ , while adding the superedge  $\{A, B\}$  to  $P$  and  $T_{AB} \setminus E_{AB}$  to  $C^-$  increases  $\varphi$  by  $1 + |T_{AB}| - |E_{AB}|$ . The above rules always choose an option resulting in a smaller increase in  $\varphi$ .

### 3.2 MoSSo-GREEDY: First Baseline

We present MoSSo-GREEDY, a baseline streaming algorithm for Problem 1, and then we point out its limitations.

**Procedure:** When an edge  $\{u, v\}$  is inserted or deleted, MoSSo-GREEDY greedily moves  $u$  and then  $v$ , while fixing the other nodes, so that the objective  $\varphi$  is minimized. That is, in terms of the introduced notions, MoSSo-GREEDY sets  $TP(u) = TN(u) = \{u\}$ , and a candidate is chosen from  $CP(y) = V$  so that  $\varphi$  is minimized.

**Limitation:** While  $|TN(u)|$  is just 1, unlike the other algorithms described below, this approach is computationally expensive as it takes all supernodes into account to find a locally best candidate. It is also likely to get stuck in a local optimum, as described below.

- **Limitation 1 (OBSTRUCTIVE OBSESSION):** MoSSo-GREEDY lacks exploration for reorganizing supernodes, and thus nodes tend to stay in supernodes that they move into in an early stage. This stagnation also prevents new nodes from moving into existing supernodes. These lead to poor compression rates in the long run.

### 3.3 MoSSo-MCMC: Second Baseline

We present MoSSo-MCMC, another streaming baseline algorithm based on randomized search. It significantly reduces the computational cost of each trial, compared to MoSSo-GREEDY, since it does not have to find the best candidate, and thus makes more trials affordable. Moreover, its randomness helps escaping from local optima and smoothly coping with changing optima. However, this approach also suffers from two drawbacks, as described later.

**Motivation:** Randomized searches based on Markov Chain Monte Carlo (MCMC) [10] have proved effective for the inference of stochastic block models (SBM) [11, 23]. We focus on an interesting relation between communities in SBM and supernodes in graph summarization. Since nodes belonging to the same community are likely to have similar connectivity, grouping them into a supernode may achieve significant reduction in  $\varphi$ . Hence, inspired by [11], we propose MoSSo-MCMC for Problem 1.

**Procedure:** In response to each change  $\{u, v\}^+$  or  $\{u, v\}^-$  in the input graph, MoSSo-MCMC performs the following steps for  $u$  (and then the exactly same steps for  $v$ ):

**Trials related to  $u$  by MoSSo-MCMC:**

- (1) Set  $TN(u) = TP(u) = N(u)$ .
- (2) For each  $y$  in  $TN(u)$ , select a candidate  $z$  from  $CP(y) = V$  through sampling according to a predefined proposal probability distribution [23].
- (3) For each  $y$ , accept the proposal (i.e., move  $y$  into  $S_z$ ) with an acceptance probability, which depends on the change in  $\varphi$ .

In Step (1), the neighbors of  $u$  are used as testing nodes since they are affected most by the input change. The  $\deg(u)$  trials can be afforded since a trial in MoSSo-MCMC is computationally cheaper than that in MoSSo-GREEDY. The proposal probability distribution and the acceptance probability used in Steps (2) and (3) are described in detail, with a pseudo code of MoSSo-MCMC, in Appendix C.

**Limitations:** MoSSo-MCMC suffers from two limitations, which are the bottlenecks of its speed and compression rates.

- **Limitation 2 (COSTLY NEIGHBORHOOD RETRIEVALS):** To process each change  $\{u, v\}^+$  or  $\{u, v\}^-$  in the input stream, MoSSo-MCMC retrieves the neighborhood of many nodes from current  $G^*$  and  $C$  (see the proof of Lemma 1 for a detailed procedure of the retrieval). Specifically, it retrieves the neighborhood of  $u$  in Step (1), and for each testing node  $y$ , it retrieves the neighborhood of at least one node to select a candidate in Step (2). That is, at least  $2 + |TN(u)| + |TN(v)| = 2 + \deg(u) + \deg(v)$  neighborhood retrievals occur. Thus, the time complexity of MoSSo-MCMC is deadly affected by growth of graphs, which may lead to the appearance of high-degree nodes and the increase of average degree [17].

- **Limitation 3 (REDUNDANT TESTS):** For proposals to be accepted, promising candidates leading to reduction in  $\varphi$  need to be sampled from the proposal probability distribution. However, the probability distribution [23], which proved successful for SBM, results in mostly rejected proposals and thus a waste of computational time.

### 3.4 MoSSo-SIMPLE: Simple Proposed Method

We present MoSSo-SIMPLE, a preliminary version of MoSSo, with three novel ideas for addressing the limitations that the baseline streaming algorithms suffer from. Then, in order to look for further improvement, we identify some limitations of MoSSo-SIMPLE.

**Procedure:** MoSSo-SIMPLE is described in Alg. 1. In response to each change  $\{u, v\}^+$  or  $\{u, v\}^-$  in the input graph, MoSSo-SIMPLE, equipped with the novel ideas described below, conducts the following steps for  $u$  (and then exactly the same steps for  $v$ ):

**Trials related to  $u$  by MoSSo-SIMPLE:**

- (1) Sample a fixed number (denoted by  $c$ ) of nodes from  $N(u)$  and use them as  $TP(u)$ .
- (2) Add each  $w \in TP(u)$  to  $TN(u)$  with probability  $\frac{1}{\deg(w)}$ .
- (3) For each  $y \in TN(u)$ , with probability  $e$  of escape, propose creating a singleton supernode  $\{y\}$ .
- (4) Otherwise, randomly select a candidate  $z$  from  $CP(y)$  where  $CP(y) = N(u)$  for every  $y \in TN(u)$ .
- (5) For each  $y$ , accept the proposal (i.e., move  $y$  to  $S_z$ ) if and only if it reduces  $\varphi$ .

**Key Ideas:** Contrary to MoSSo-MCMC, for each change  $\{u, v\}^+$  or  $\{u, v\}^-$ , MoSSo-SIMPLE (1) extracts  $TN(u)$  from  $TP(u)$  probabilistically depending the degrees of nodes, (2) limits  $CP(y)$  to  $N(u)$  for every testing node  $y \in TP(u)$ , and (3) occasionally separates  $y$  from  $S_y$  and create a singleton supernode  $\{y\}$ . As shown in

Sect. 4, these ideas enable MoSSo-SIMPLE to significantly outperform MoSSo-MCMC, as well as MoSSo-GREEDY, in terms of speed and compression rates. Below, we describe each idea in detail.

- **CAREFUL SELECTION (1)**: When forming  $TN(u)$ , MoSSo-SIMPLE first samples with replacement a fixed number of nodes from  $N(u)$  and construct  $TP(u)$  using them. Then, it adds each sampled node  $w \in TP(u)$  to  $TN(u)$  with probability  $1/\deg(w)$ . In practice, high-degree nodes tend to have unique connectivity, and thus they tend to form singleton supernodes. Therefore, moving them rarely leads to the reduction of  $\phi$ . However, high-degree nodes are frequently contained in  $TP(u)$  since edge changes adjacent to any neighbor  $u$  put the high-degree nodes into  $TP(u)$ . Moreover, once they are chosen as testing nodes, computing the change in  $\phi$  and updating the optimal encoding are computationally expensive since they have many neighbors. By probabilistically filtering out high-degree nodes when forming  $TN(u)$ , MoSSo-SIMPLE significantly reduces redundant and computationally expensive trials and thus partially addresses Limitation 3 (i.e., REDUNDANT TESTS).

- **CORRECTIVE ESCAPE**: Instead of always finding a candidate from  $CP(y)$ , it separates  $y$  from  $S_y$  and creates a singleton supernode  $\{y\}$  with probability  $e \in [0, 1]$ . By injecting flexibility to the formation of a summary graph, this idea, which we call CORRECTIVE ESCAPE, helps supernodes to be reorganized in different and potentially better ways in the long run. Therefore, this idea addresses Limitation 1 and yields significant improvement in compression.

- **FAST RANDOM (1)**: By limiting  $CP(y)$  to  $N(u)$  for every testing node  $y \in TN(u)$ , MoSSo-SIMPLE reduces the number of required neighborhood retrievals and thus partially addresses Limitation 2 (i.e., COSTLY NEIGHBORHOOD RETRIEVALS). For each input change, while MoSSo-MCMC repeats neighborhood retrievals  $2 + \deg(u) + \deg(v)$  times, MoSSo-SIMPLE only retrieves  $N(u)$  and  $N(v)$ . Moreover, since  $N(u)$  still contains promising candidates, limiting  $CP(y)$  to  $N(u)$  does not impair the compression rates as shown empirically in Sect. 4.

**Limitations:** Although the above ideas successfully mitigate the limitations, there remain issues to be resolved.

- **Limitation 2 (COSTLY NEIGHBORHOOD RETRIEVALS)**: While MoSSo-SIMPLE reduces the number of neighborhood retrievals to two per each input change, the retrievals still remain as a scalability bottleneck. As analyzed in Lemma 1, retrieving the neighborhood of a node from current  $G^*$  and  $C$  takes  $O(\deg)$  time on average<sup>2</sup>, where  $\deg = \frac{2|E|}{|V|}$  is the average degree in the input graph. However, it is well known that lots of real-world graphs are densified over time [17]. Specifically, the number of edges increases super-linearly in the number of nodes, leading to the growth of  $\deg$  over time. Hence, full neighborhood retrievals pose a huge threat to scalability.

- **Limitation 3 (REDUNDANT TESTS)**: While MoSSo-SIMPLE uses the degree of nodes to reduce redundant trials, which lead to rejected proposals, it does not fully make use of structural information around input nodes but simply draws a random candidate from  $N(u)$ . Careful selection of candidates based on the structural information can be desirable to further reduce the number of redundant trials and thus to achieve concise summarization rapidly.

<sup>2</sup>To retrieve the neighborhood  $N(y)$  of  $y$ , we need to collect all its neighbors in  $C^+$  (i.e.,  $\{v \in V \mid \{v, y\} \in C^+\}$ ) and all nodes in the neighboring supernodes of  $S_y$  (i.e.,  $\{v \in V \mid v \in S_u, S_u \in N(S_y)\}$  where  $N(S_y) := \{S_u \in S \mid \{S_u, S_y\} \in P\}$ ). Then, we need to filter out all its neighbors in  $\{v \in V \mid \{u, y\} \in C^-\}$  (see Sect. 2.1).

**Algorithm 1: MoSSo AND MoSSo-SIMPLE:** proposed algorithms for Problem 1.

---

► The red and blue lines are executed only in MoSSo and MoSSo-SIMPLE, respectively.

► The black lines are executed in both algorithms.

**Input:** summary graph:  $G_t^*$ , edge corrections:  $C_t$ , input change:  $\{src, dst\}^+$  or  $\{src, dst\}^-$  escape probability:  $e$ , sample number:  $c$ , **coarse clusters:  $R$**

**Output:** summary graph:  $G_{t+1}^*$ , edge corrections:  $C_{t+1}$

```

1 update  $R$  in response to the input change
2 foreach  $u$  in  $\{src, dst\}$  do
3    $TP(u) \leftarrow \text{GETRANDOMNEIGHBOR}(c)$  ► see Alg. 2
4    $TP(u) \leftarrow$  randomly chosen  $c$  neighbors from  $N(u)$ 
5   foreach  $w$  in  $TP(u)$  do
6     put  $w$  into  $TN(u)$  with probability  $1/\deg(w)$ 
7   foreach  $y$  in  $TN(u)$  do
8     sample  $X \sim \text{uniform}(0, 1)$ 
9     if  $X \leq e$  then
10      temporarily create a new supernode  $S_y = \{y\}$ 
11    else
12       $R(y) \leftarrow$  the coarse cluster containing  $y$ 
13       $CP(y) \leftarrow TP(u) \cap R(y)$  or  $CP(y) \leftarrow N(u)$ 
14      randomly choose a node  $z$  from  $CP(y)$ 
15      temporarily move  $y$  into  $S_z$ 
16      compute  $\Delta\phi$  (i.e., change in  $\phi$ ) for the proposal
17      if  $\Delta\phi \leq 0$  then
18        accept the proposal and update  $G_t^*, C_t$ 
19        ► see Sect. 3.1 for optimal encoding
20  $G_{t+1}^*, C_{t+1} \leftarrow G_t^*, C_t$ 
21 return  $G_{t+1}^*, C_{t+1}$ 

```

---

### 3.5 MoSSo: Full-Fledged Proposed Method

We present MoSSo, the full-fledged version of our algorithms. To overcome the aforementioned drawbacks of MoSSo-SIMPLE, MoSSo employs (1) coarse clustering for careful candidate selection and (2) GETRANDOMNEIGHBOR, a novel sampling method instead of full neighborhood retrievals. Equipped with these ideas, MoSSo achieves near-constant processing time per change and compression rates even comparable to state-of-the-art batch algorithms.

**Procedure:** A pseudo code of MoSSo is provided in Alg. 1. In response to each change  $\{u, v\}^+$  or  $\{u, v\}^-$ , MoSSo conducts the following steps for  $u$  (and then exactly the same steps for  $v$ ):

#### Trial related to $u$ by MoSSo:

- (1) Update **coarse clusters** in response to the change.
- (2) Sample a fixed number (denoted by  $c$ ) of nodes from  $N(u)$ , **without retrieving all  $N(u)$** , and use them as  $TP(u)$ .
- (3) Add each  $w \in TP(u)$  to  $TN(u)$  with probability  $\frac{1}{\deg(w)}$ .
- (4) For each  $y \in TN(u)$ , with probability  $e$  of escape, propose creating a singleton supernode  $\{y\}$ .
- (5) Otherwise, randomly select a candidate  $z$  from  $CP(y)$  where  $CP(y) = TP(u) \cap R(y)$  and  $R(y)$  is the **coarse cluster** containing  $y$ , for every  $y \in TN(u)$ .
- (6) For each  $y$ , accept the proposal (i.e., move  $y$  to  $S_z$ ) if and only if it reduces  $\phi$ .

**Key Ideas:** The gist of MoSSo consists of two parts: (1) rapidly and uniformly sampling neighbors from  $N(u)$  without retrieving the entire  $N(u)$  from  $G^*$  and  $C$  and (2) using an online coarse clustering to narrow down  $CP(y)$ . Below, we describe each idea in detail.

• **FAST RANDOM (2):** As explained in Limitation 2 (Sect. 3.4), for scalability, it is inevitable to devise a neighborhood sampling method less affected by the average degree, which tends to increase over time. Thus, we come up with GETRANDOMNEIGHBOR, described in Alg. 2. It is an MCMC method for rapidly sampling nodes from  $N(u)$  in an unbiased manner without retrieving the entire  $N(u)$ . After obtaining a sufficient number of neighbors by GETRANDOMNEIGHBOR, MoSSo limits  $TP(u)$  to the sampled neighbors.

Assume that the neighborhood in  $C^+$ ,  $C^-$  and  $P$  of each node is stored in a hash table, and let  $N(S_u) := \{S_v \in S \mid \{S_u, S_v\} \in P\}$  be the set of neighboring supernodes of a supernode  $S_u$ . Then,  $v \in N(u)$  can be checked rapidly as follows:

**Checking  $v \in N(u)$  on  $G^*$  and  $C$ :**

- (1) If  $v \in C^-(u)$ , then  $v \notin N(u)$ .
- (2) Else if  $v \in C^+(u)$  or  $S_v \in N(S_u)$ , then  $v \in N(u)$ .
- (3) Else  $v \notin N(u)$ .

The neighbors of  $u$  are divided into two disjoint sets: (1) those in  $C^+(u)$  and (2) those in any supernode in  $N(S_u)$ . Then, we can uniformly sample a neighbor of  $u$  by uniformly sampling a node either from the first set (with probability  $\frac{|C^+(u)|}{\deg(u)}$ ) or from the second set (with probability  $1 - \frac{|C^+(u)|}{\deg(u)}$ ). Since the first set is already materialized (by our assumption), uniform sampling from it is straightforward. The remaining challenge is to uniformly sample a node from the second set without materializing the second set. This challenge is formulated in Problem 2, where we denote  $N(S_u)$  by  $\{S_1, \dots, S_k\}$ .

**PROBLEM 2 (SUBPROBLEM 1 FOR FAST RANDOM (2)).** *We have  $k$  disjoint supernodes  $S_1, \dots, S_k$  with  $N_i := N(u) \cap S_i$  for each  $i$ . For large  $k$ , it is computationally expensive to obtain  $\bigcup_{i=1}^k N_i$ . However, we can rapidly check whether a given node is contained in  $\bigcup_{i=1}^k N_i$ . How can we rapidly and uniformly draw nodes from  $\bigcup_{i=1}^k N_i$ ?*

**Our solution:** Instead of uniform sampling from  $\bigcup_{i=1}^k N_i$ , GETRANDOMNEIGHBOR, described in Alg. 2, samples a node from  $\bigcup_{i=1}^k S_i$  and retries sampling if the node is not in  $\bigcup_{i=1}^k N_i$ . To this end, it randomly selects a supernode  $S_i$  with probability

$$\pi(S_i \text{ is selected}) := |S_i| / (|S_1| + \dots + |S_k|), \quad (2)$$

and then draws a random node from the selected supernode. If the node is not in  $\bigcup_{i=1}^k N_i$ , then this procedure is repeated again from beginning. It is guaranteed that this sampling scheme draws each node in  $\bigcup_{i=1}^k N_i$  uniformly with probability  $1/N$ , where  $N := \sum_{i=1}^k |N_i|$ , as shown in Thm. 1 in Sect. 3.6.

In the above solution, when sampling supernodes according to Eq. (2), it is desirable to avoid computing  $|S_i|$  for every  $i \in \{1, \dots, k\}$ , since  $k$  can be large, as formulated in Problem 3.

**PROBLEM 3 (SUBPROBLEM 2 FOR FAST RANDOM (2)).** *How to sample supernodes according to Eq. (2) without computing  $|S_i|$  for each  $i$ ?*

**Our solution:** GETRANDOMNEIGHBOR employs MCMC, which basically constructs a Markov chain asymptotically equal to Eq (2). Specifically, a supernode  $S_p$  is proposed uniformly at random among  $k$  supernodes, and then it replaces a previously sampled supernode, which is denoted by  $S_n$ , with the following acceptance probability:

$$\min \left( 1, \frac{\pi(S_p \text{ is selected})}{\pi(S_n \text{ is selected})} \right) = \min \left( 1, \frac{|S_p|}{|S_n|} \right).$$

The soundness is shown in Thm. 2 in Sect. 3.6.

---

#### Algorithm 2: GETRANDOMNEIGHBOR for MoSSo.

---

**Input:** summary graph:  $G_t^*$ , edge corrections:  $C_t$ ,  
input node:  $u$ , sample number:  $c$

**Output:** list of  $c$  random neighbors from  $N(u)$

```

1  $V_r \leftarrow []$ ,  $P \leftarrow N(S_u)$ ,  $C_p \leftarrow C_t^+(u)$ ,  $C_m \leftarrow C_t^-(u)$ 
2  $S_n \leftarrow$  random supernode in  $P$ 
3 while  $|V_r| < c$  do
4   sample  $X_1 \sim \text{uniform}(0, 1)$ 
5   if  $X_1 \leq |C_p|/\deg(u)$  then
6     put a random node from  $C_p$  into  $V_r$ 
7   else
8     while do
9        $S_p \leftarrow$  a random supernode in  $P$ 
10      sample  $X_2 \sim \text{uniform}(0, 1)$ 
11      if  $X_2 \leq \min(1, |S_p|/|S_n|)$  then
12         $S_n \leftarrow S_p$ 
13       $w \leftarrow$  a random node in  $S_n$ 
14      if  $w \notin C_m$  then
15        add  $w$  to  $V_r$ 
16      break
17 return  $V_r$ 
```

---

Both solutions are combined in Alg. 2, which describes the entire process for sampling  $c$  neighbors from  $N(u)$ . We analyze its soundness and time complexity in Sect. 3.6.

• **CAREFUL SELECTION (2):** To choose candidates leading to a significant reduction in  $\phi$ , MoSSo uses coarse clusters, each of which consists of nodes with similar connectivity. Specifically, in MoSSo, the candidate pool  $CP(y)$  of each testing node  $y$  consists only of nodes belonging to the same cluster of  $y$ . The coarse clusters are distinct from supernodes, which can be thought as fine clusters.

Any incremental graph clustering methods [22, 29, 33, 34] can be used for coarse clustering. We especially use min-hashing [5] as it is fast with desirable theoretical properties: the probability that two nodes belong to the same cluster is proportional to the jaccard similarity of their neighborhoods. Moreover, clusters grouped by min-hashing can be updated rapidly in response to changes.

By employing the above ideas, MoSSo processes each change in near-constant time and shows outstanding compression, as shown theoretically in the next subsection and empirically in Sect. 4.

### 3.6 Theoretical Analysis

We present a theoretical analysis of MoSSo (Alg. 1). We first prove the soundness and time complexity of GETRANDOMNEIGHBOR (Alg. 2), which the performance of MoSSo relies on. Then, we analyze the overall time complexity of MoSSo. Lastly, we show that MoSSo requires sub-linear memory during the incremental summarization process. **All proofs are provided in Appendix B.**

**3.6.1 Soundness of GETRANDOMNEIGHBOR (Alg. 2).** We show that GETRANDOMNEIGHBOR performs uniform sampling by proving Thms. 1 and 2 that our solutions in Sect. 3.5 solve Problems 2 and 3.

**THEOREM 1 (UNBIASEDNESS OF OUR SOLUTION FOR PROBLEM 2).** *Our solution for Problem 2 draws a uniformly random node from  $\bigcup_{i=1}^k N_i$ . That is, each node is sampled with probability  $1/\sum_{i=1}^k |N_i|$ .*

**THEOREM 2 (SOUNDNESS OF OUR SOLUTION FOR PROBLEM 3).** *Our solution for Problem 3, which is an MCMC method equipped with (a) the proposal distribution  $P(S_j|S_i) = 1/k$  for any  $i, j$  and (b) the*

acceptance probability  $A(S_j, S_i) = \min(1, |S_j|/|S_i|)$  for any move from  $S_i$  to  $S_j$ , asymptotically simulates  $\pi$  in Eq. (2).

**3.6.2 Time Complexity of GETRANDOMNEIGHBOR (Alg. 2).** Before proving the time complexity of GETRANDOMNEIGHBOR, which is a core building block of MoSSo, we first prove the time complexity of retrieving the neighborhood of an input node from the outputs (i.e.,  $G^*$  and  $C$ ) in Lemma 1. We let  $C^+(u) := \{v \in V \mid \{u, v\} \in C^+\}$  and  $C^-(u) := \{v \in V \mid \{u, v\} \in C^-\}$  be the sets of neighbors of  $u$  in  $C^+$  and  $C^-$ , respectively, and we let  $N(A) := \{B \in S \mid \{A, B\} \in P\}$  be the set of neighboring supernodes of the supernode  $A$ .

**LEMMA 1 (AVERAGE TIME COMPLEXITY OF RETRIEVING NEIGHBORHOOD).** *The average-case time complexity of retrieving the neighborhood of a node is  $O(\deg)$ , where  $\deg$  is the average degree.*

As discussed in Limitation 2 in Sect. 3.4, Lemma 1 motivates us to design GETRANDOMNEIGHBOR, which is much faster than retrieving the entire neighborhood, as formulated in Thm. 3.

**THEOREM 3 (AVERAGE TIME COMPLEXITY OF GETRANDOMNEIGHBOR).** *Assume the neighborhood in  $C^+$ ,  $C^-$  and  $P$  of each node is stored in a hash table. The average-time complexity of Alg. 2 for each node  $u$  is  $O(c \cdot (1 + |C^-(u)|/\deg(u)))$ .*

The average time complexity of GETRANDOMNEIGHBOR called by MoSSo becomes constant, under a realistic assumption [1] that an initial graph evolves under  $\{e_t\}_{t=0}^\infty$  where  $e_t$  takes place on a node with probability proportional to the degree of the node. That is, higher-degree nodes are more likely to go through changes in their neighbors. This result is shown in Corollary 1 based on Lemma 2.

**LEMMA 2.** *Let  $X$  be a discrete random variable whose domain is  $\{\frac{a_u}{b_u}\}_{u \in V}$ , where  $a_u = |C^-(u)|$  and  $b_u = \deg(u)$  for each node  $u$  and the probability mass is proportional to  $\deg(u)$ . Then,  $E[X] \leq 1$ .*

**COLLORARY 1 (AVERAGE TIME COMPLEXITY OF GETRANDOMNEIGHBOR UNDER PREFERENTIAL ATTACHMENT [1]).** *The average-case time complexity of GETRANDOMNEIGHBOR called by Alg. 1 is  $O(c)$  under the assumption that each edge change  $e_t$  takes place on nodes with probability proportional to the degree of the nodes.*

**3.6.3 Overall Time Complexity of MoSSo (Alg. 1).** In our analysis below, we let  $SN(A) := \{B \in S : \exists (u, v) \in A \times B, \text{ s.t. } \{u, v\} \in E\}$  be the set of supernodes connected to  $A$  by at least one edge. We also let  $E_{AB} := \{\{u, v\} \in E \mid u \in A, v \in B, u \neq v\}$  and  $T_{AB} := \{\{u, v\} \mid u \in A, v \in B, u \neq v\}$  be the sets of existing and potential edges between supernodes  $A$  and  $B$ , respectively.

**Computing Savings in the Objective  $\varphi$  (Eq. (1)).** Suppose that we move a node  $y$  from a supernode  $S_y$  to a supernode  $S_z$ . Then, the previous optimal encoding (see Sect. 3.1) between  $S_y$  and each of  $SN(S_y)$  and between  $S_z$  and each of  $SN(S_z)$  can be affected, while that between the other supernodes should remain the same. Hence, only  $SN(S_y)$  and  $SN(S_z)$  are of concern when computing the saving in the objective  $\varphi$  (i.e., Eq (1)) induced by the move of  $y$ . Thus, the time complexity of this task is  $O(|SN(S_y)| + |SN(S_z)|)$ .

**Updating Optimal Encoding:** The computation of the saving determines whether a move is accepted or not. Once the move is accepted, the previous optimal encoding should be updated in response to the move. As explained in Sect. 3.1, the edges between two supernodes  $A$  and  $B$  are optimally encoded either as (1)  $|E_{AB}|$

**Table 3: Real-world graph streams used in our experiments.**

| Name                              | #Nodes       | #Insertions   | #Deletions  | Summary             |
|-----------------------------------|--------------|---------------|-------------|---------------------|
| Insertion-only (IO) graph streams |              |               |             |                     |
| Protein (PR)                      | 6, 229       | 146, 160      | -           | Protein Interaction |
| Email-Enron (EN)                  | 86, 977      | 297, 456      | -           | Email               |
| Facebook (FB)                     | 61, 095      | 614, 797      | -           | Friendship          |
| Web-EU-05 (EU)                    | 862, 664     | 16, 138, 468  | -           | Hyperlinks          |
| Hollywood (HW)                    | 1, 985, 306  | 114, 492, 816 | -           | Collaboration       |
| Web-UK-02 (UK)                    | 18, 483, 186 | 261, 787, 258 | -           | Hyperlinks          |
| Fully-dynamic (FD) graph streams  |              |               |             |                     |
| DBLP (DB)                         | 317, 080     | 1, 049, 866   | 116, 042    | Coauthorship        |
| YouTube (YT)                      | 1, 134, 890  | 2, 987, 624   | 331, 305    | Friendship          |
| Skitter (SK)                      | 1, 696, 415  | 11, 095, 298  | 1, 233, 226 | Internet            |
| LiveJournal (LJ)                  | 3, 997, 962  | 34, 681, 189  | 3, 854, 423 | Friendship          |

edges in  $C^+$  or as (2) the superedge  $\{A, B\}$  with  $|T_{AB}| - |E_{AB}|$  edges in  $C^-$ . In the latter case,  $|T_{AB}| - |E_{AB}| + 1 \leq |E_{AB}|$  holds in the optimal encoding. Hence, in both cases, the number of references needed to update the optimal encoding between each supernode pair  $\{A, B\}$  is  $O(|E_{AB}|)$ . Due to the reason discussed in the previous paragraph, the total worst-case time complexity is

$$O\left(\sum_{U \in SN(S_y)} |E_{S_y U}| + \sum_{U \in SN(S_z)} |E_{S_z U}| \right) = O\left(\sum_{u \in S_y} \deg(u) + \sum_{u \in S_z} \deg(u)\right).$$

**Scalability in Practice:** In practice, the above two subtasks do not harm the scalability of MoSSo, which proceeds to trials only with probability  $\frac{1}{\deg}$  and updates the optimal encoding only if a move is accepted. We show in Appendix. A.1 that the runtime of MoSSo in practice increases proportionally to the number of samples (i.e.,  $c$ ) per input change, as expected from Corollary 1. For a fixed  $c$ , MoSSo processes each change in **near-constant time**, regardless of the growth of the input graph, as we show empirically in Sect. 4.4.

**3.6.4 Space Complexity of MoSSo (Alg. 1).** The sub-linear space complexity of MoSSo proved in Thm. 4 implies that the entire input graph of size  $O(|V| + |E|)$  does not have to be maintained in memory during the incremental summarization process.

**THEOREM 4 (SPACE COMPLEXITY OF MoSSo).** *The space complexity of Algorithm 1 is  $O(|V| + |P| + |C^+| + |C^-|)$ .*

## 4 EXPERIMENTS

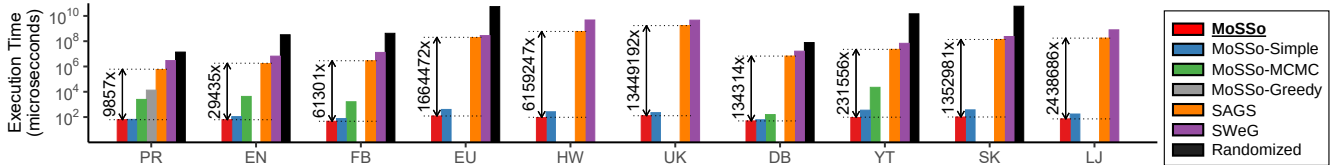
We review our experiments for answering the following questions:

- Q1. **Speed:** How fast is MoSSo, compared to the baseline streaming algorithms and the best batch algorithms?
- Q2. **Compression Ratio:** How compact are the outputs of MoSSo, compared to those obtained by the best competitors?
- Q3. **Scalability:** How does the runtime of MoSSo grow as the input graph grows?
- Q4. **Performance Analysis (Appendix A):** How do graph properties and parameter values affect the performance of MoSSo?

### 4.1 Experimental Settings

**Machine:** We performed our experiments on a desktop with a 3.7GHz Intel i5-9600k CPU and 64GB memory.

**Dataset:** We used ten real-world graphs listed in Table 3. In every graph, we ignored the direction of all edges and removed both self-loops and multiple edges. From the graphs, we generate both insertion-only and fully dynamic graph streams with deletions. We generated an insertion-only graph stream by listing the insertions



**Figure 4: MoSSo is fast.** It processed each change in the input graph up to 7 orders and 2 orders of magnitude faster than the fastest batch algorithm and the streaming baselines (MoSSo-Greedy and MoSSo-MCMC), respectively. The baseline streaming algorithms and RANDOMIZED ran out of time ( $> 24$  hours) on some large graphs and do not appear in the corresponding plots.

of all edges in a graph. We either sorted the insertions by their timestamps (if they exist) or randomly ordered them (otherwise). For a fully-dynamic graph stream, we listed the insertions of all edges in a graph after randomly ordering them. Then, for each edge, we created the deletion of it with probability 0.1 and located the deletion in a random position after the insertion of the edge.

**Implementation:** We implemented the following lossless graph-summarization algorithms in OpenJDK 12: (a) MoSSo (proposed) with  $e=0.3$ ,  $c=120$ ; (b) MoSSo-Simple (proposed) with  $e=0.3$ ,  $c=120$ ; (c) MoSSo-Greedy (streaming baseline), (d) MoSSo-MCMC (streaming baseline) with  $\beta = 10$ ; (e) SWeG [27] (batch baseline) with  $\#Threads=1$ ,  $T=20$ ,  $\epsilon=0$ ; (f) SAGS [13] (batch baseline) with  $h=30$ ,  $b=10$ ,  $p=0.3$ ; and (g) RANDOMIZED [21] (batch baseline).

**Evaluation Metric:** Given a summary graph  $G^* = (S, P)$  and edge corrections  $C = (C^+, C^-)$  of a graph  $G = (V, E)$ , we used

$$(|P| + |C^+| + |C^-|)/|E| \quad (3)$$

as the *compression ratio*. In Eq. (3), the numerator is the objective  $\varphi$ , and the denominator is a constant, given an input graph  $G$ . Eq. (3) and runtime were averaged over 3 trials unless otherwise stated.

## 4.2 Q1. Speed (Fig. 4)

To evaluate the speed of MoSSo, we measured the runtime of all considered algorithms. Specifically, for batch algorithms, we measured the time taken for summarizing each dataset. For streaming algorithms, we measured time taken for processing all the changes in each dataset, and then we divided it by the number of changes to compute the time taken for processing each change.

**MoSSo was significantly faster than its competitors.** As seen in Fig. 4, it processed each change up to 7 orders of magnitude faster than running the fastest batch algorithm. The gap between MoSSo and the batch algorithms was wide in large datasets. Moreover, MoSSo was up to 2 orders of magnitude faster than the baseline streaming algorithms, and the gap increased in larger datasets, where the baseline method ran out of time. The two versions of MoSSo showed comparable speed in small datasets, while MoSSo was faster than MoSSo-Simple in large datasets.

## 4.3 Q2. Compression Ratio (Fig. 5)

To test the effectiveness of MoSSo, we measured the compactness of outputs obtained by all considered algorithms using the compression ratio, defined in Eq (3). For streaming algorithms, we tracked the changes in the compression ratio while each input graph stream evolved over time. For batch algorithms, we ran them when 20%, 40%, 60%, 80% and 100% of the changes in each dataset arrived.

**MoSSo achieved the best compression ratios among the streaming algorithms**, as seen in Fig. 5. Moreover, the compression ratio of MoSSo was even comparable to those of the best batch algorithms. Between the two versions of MoSSo, MoSSo consistently showed better compression ratios than MoSSo-Simple.

## 4.4 Q3. Scalability (Fig. 1(c))

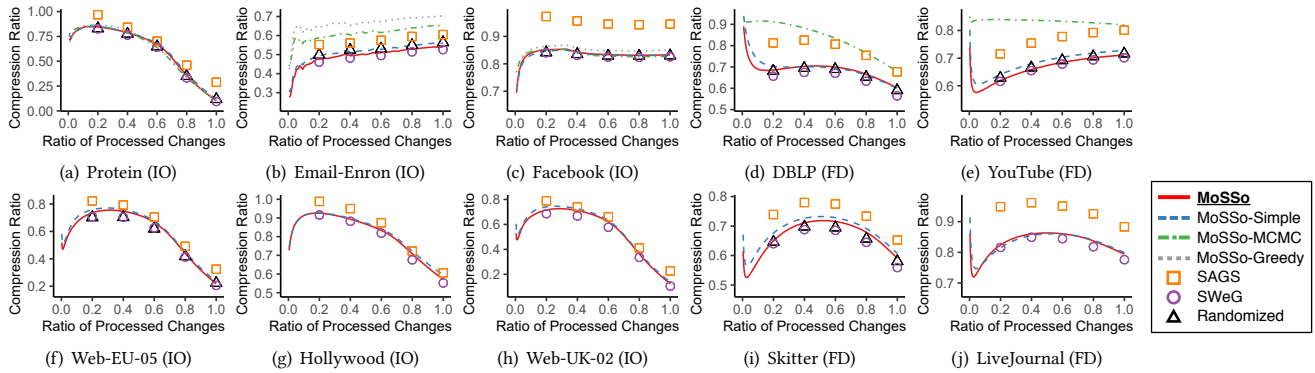
To evaluate the scalability of MoSSo, we measured its runtime in the Web-EU-05 dataset. As seen in Fig. 1(c), the accumulated runtime of MoSSo was near-linear in the number of processed changes, implying that **MoSSo processed each change in near-constant time**. Note that we ignored time taken to wait for input changes to arrive. More extensive results can be found at Appendix A.3.

## 5 RELATED WORKS

Graph summarization has been studied in a number of settings, and an extensive survey can be found in [18]. We focus on studies directly related to our work: (1) lossless summarization of static graphs, and (2) lossy and lossless summarization of dynamic graphs.

**Lossless summarization of static graphs:** Lossless summarization algorithms on static graphs output a summary graph with edge corrections. GREEDY [21] examines all pairs of supernodes that are 2-hops away, and among the pairs, it repeatedly merges the pair leading to the largest saving in the objective  $\varphi$ . RANDOMIZED [21] speeds up the process by randomly picking one supernode and searching only for the second supernode that leads to the largest saving in  $\varphi$ , when being merged with the first one. To further reduce the time complexity, SAGS [13] selects two supernodes to be merged based on locality sensitive hashing. While these algorithms fail to strike a balance between compression and time complexity, a parallel algorithm SWeG [27] succeeds in improving both parts. Note that these batch algorithms are not designed to address changes in the input graph and should be rerun from scratch to reflect such changes. Lossy variants of the graph summarization problem were also explored in [15, 16, 21, 24, 27].

**Summarization of dynamic graphs:** Since the dynamic nature of graphs makes lossless summarization more challenging, previous studies of dynamic graph summarization have focused largely on lossy summarization for query efficiency [9, 30], social influence analysis [19, 20], temporal pattern [31], etc. They aim to summarize: either the current snapshot of the input graph [9, 12, 30, 32] or temporal patterns in the entire history of the growth of the input graph [31]. For the former, existing incremental algorithms [9, 12, 30, 32] produce lossy summary for query efficiency in a form of graph sketch (i.e., an adjacency matrix smaller than the original one). Especially, TCM [30] and its variant GSS [9] compress the original graph  $G = (V, E)$  into a graph sketch  $G_s$  by using a hash function  $h(\cdot)$  through mapping a node  $v \in V$  into a node  $h(v)$  in  $G_v$  and mapping an edge  $\{u, v\} \in E$  into  $\{h(u), h(v)\}$  in  $G_v$ . For the latter,  $kC$  &  $k\mu$  [31] lossily compress the entire history of a dynamic graph to find temporal patterns. While TIMECRUNCH [26] is lossless, it is a batch algorithm for summarizing the entire growth history of a dynamic graph. To the best of our knowledge, MoSSo is the first incremental algorithm for summarizing the current snapshot of a fully dynamic graph in a lossless manner.



**Figure 5: MoSSo is ‘any time’ with compact representations.** MoSSo achieved the best compression ratios among streaming algorithms, which always maintain the summarized representation of the input graph. Moreover, MoSSo achieved compression ratios comparable to that of the best batch algorithms, which were rerun repeatedly. The baseline streaming algorithms and RANDOMIZED ran out of time ( $> 24$  hours) on some large graphs and so, do not appear in the corresponding plots.

## 6 CONCLUSIONS

We propose MoSSo, a simple but fast and effective incremental algorithm for lossless summarization of fully-dynamic graphs. MoSSo is based on several novel ideas, including CORRECTIVE ESCAPE, FAST RANDOM, and CAREFUL SELECTION, and they lead to significant improvement in speed and compression rates. We empirically and theoretically show that MoSSo has the following strengths:

- **Fast and ‘any time’:** In response to each change, MoSSo updates its lossless summary up to **7 orders of magnitude faster** than the fastest batch algorithm (Fig. 4). The update time remains constant while the input graph grows (Fig. 1(c)).
- **Scalable:** MoSSo successfully summarizes a fully dynamic graph with up to 0.3 billion edges, without having to maintain the original graph in memory (Thm. 4).
- **Effective:** MoSSo shows compression rates comparable to state-of-the-art batch algorithms (Fig. 5).

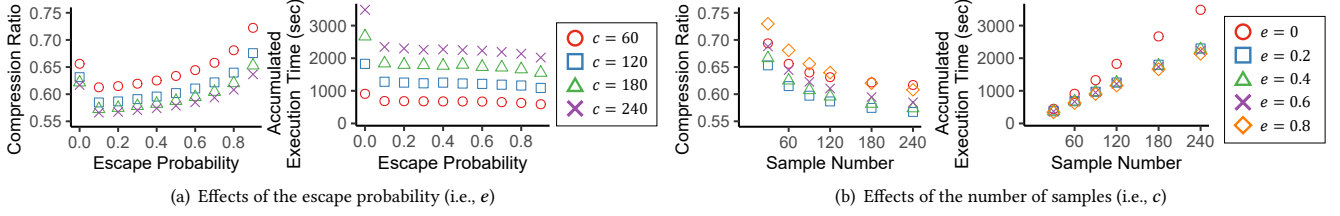
**Reproducibility:** The code and datasets used in the paper are available at <http://dmlab.kaist.ac.kr/mosso/>.

## Acknowledgements

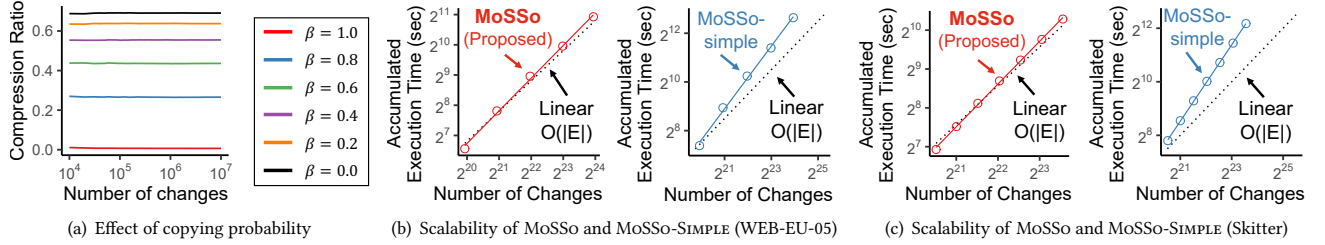
This work was supported by National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. NRF-2019R1F1A1059755) and Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2019-0-00075, Artificial Intelligence Graduate School Program (KAIST)).

## REFERENCES

- [1] Réka Albert and Albert-László Barabási. 2002. Statistical mechanics of complex networks. *Reviews of modern physics* 74, 1 (2002), 47.
- [2] Maham Anwar Beg, Muhammad Ahmad, Arif Zaman, and Imdadullah Khan. 2018. Scalable Approximation Algorithm for Graph Summarization. In *PAKDD*.
- [3] Eric Temple Bell. 1938. The iterated exponential integrals. *Annals of Mathematics* (1938), 539–557.
- [4] Paolo Boldi and Sebastiano Vigna. 2004. The webgraph framework I: compression techniques. In *WWW*.
- [5] Andrei Z Broder, Moses Charikar, Alan M Frieze, and Michael Mitzenmacher. 2000. Min-wise independent permutations. *JCSS* 60, 3 (2000), 630–659.
- [6] Gregory Buehrer and Kumar Chellapilla. 2008. A scalable pattern mining approach to web graph compression with communities. In *WSDM*.
- [7] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, Michael Mitzenmacher, Alessandro Panconesi, and Prabhakar Raghavan. 2009. On compressing social networks. In *KDD*.
- [8] Laxman Dhulipala, Igor Kabiljo, Brian Karrer, Giuseppe Ottaviano, Sergey Pupyrev, and Alon Shalita. 2016. Compressing graphs and indexes with recursive graph bisection. In *KDD*.
- [9] Xiangyang Gou, Lei Zou, Chenxingyu Zhao, and Tong Yang. 2019. Fast and Accurate Graph Stream Summarization. In *ICDE*.
- [10] W Keith Hastings. 1970. Monte Carlo sampling methods using Markov chains and their applications. (1970).
- [11] Edward Kao, Vijay Gadepally, Michael Hurley, Michael Jones, Jeremy Kepner, Sanjeev Mohindra, Paul Monticciolo, Albert Reuther, Siddharth Samsi, William Song, et al. 2017. Streaming graph challenge: Stochastic block partition. In *HPEC*.
- [12] Arijit Khan and Charu Aggarwal. 2017. Toward query-friendly compression of rapid graph streams. *SNAM* 7, 1 (2017), 23.
- [13] Kifayat Ullah Khan, Waqas Nawaz, and Young-Koo Lee. 2015. Set-based approximate approach for lossless graph summarization. *Computing* 97, 12 (2015), 1185–1207.
- [14] Jon M Kleinberg, Ravi Kumar, Prabhakar Raghavan, Sridhar Rajagopalan, and Andrew S Tomkins. 1999. The web as a graph: measurements, models, and methods. In *COCOON*.
- [15] Kyuhan Lee, Hyeonsoo Jo, Jihoon Ko, Sungsu Lim, and Kijung Shin. 2020. SSumM: Sparse Summarization of Massive Graphs. In *KDD*.
- [16] Kristen LeFevre and Evimaria Terzi. 2010. GraSS: Graph structure summarization. In *SDM*.
- [17] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. 2007. Graph evolution: Densefication and shrinking diameters. *TKDD* 1, 1 (2007), 2.
- [18] Yike Liu, Tara Safavi, Abhilash Dighe, and Danai Koutra. 2018. Graph Summarization Methods and Applications: A Survey. *CSUR* 51, 3 (2018), 62.
- [19] Michael Mathioudakis, Francesco Bonchi, Carlos Castillo, Aristides Gionis, and Antti Ukkonen. 2011. Sparsification of influence networks. In *KDD*.
- [20] Yasir Mehmood, Nicola Barbieri, Francesco Bonchi, and Antti Ukkonen. 2013. Csi: Community-level social influence analysis. In *ECML/PKDD*.
- [21] Saket Navlakha, Rajeev Rastogi, and Nisheeth Shrivastava. 2008. Graph summarization with bounded error. In *SIGMOD*.
- [22] Huazhong Ning, Wei Xu, Yun Chi, Yihong Gong, and Thomas Huang. 2007. Incremental spectral clustering with application to monitoring of evolving blog communities. In *SDM*.
- [23] Tiago P Peixoto. 2014. Efficient Monte Carlo and greedy heuristic for the inference of stochastic block models. *Physical Review E* 89, 1 (2014), 012804.
- [24] Matteo Riondato, David Garcia-Soriano, and Francesco Bonchi. 2017. Graph summarization with quality guarantees. *DMKD* 31, 2 (2017), 314–349.
- [25] Jorma Rissanen. 1978. Modeling by shortest data description. *Automatica* 14, 5 (1978), 465–471.
- [26] Neil Shah, Danai Koutra, Tianmin Zou, Brian Gallagher, and Christos Faloutsos. 2015. Timecrunch: Interpretable dynamic graph summarization. In *KDD*.
- [27] Kijung Shin, Amol Ghoting, Myunghwan Kim, and Hema Raghavan. 2019. Sweg: Lossless and lossy summarization of web-scale graphs. In *WWW*.
- [28] Julian Shun, Laxman Dhulipala, and Guy E Blelloch. 2015. Smaller and faster: Parallel processing of compressed graphs with Ligra+. In *DCC*.
- [29] Mansoureh Takaffoli, Reihaneh Rabbany, and Osmar R Zaiane. 2013. Incremental local community identification in dynamic social networks. In *ASONAM*.
- [30] Nan Tang, Qing Chen, and Prasenjit Mitra. 2016. Graph stream summarization: From big bang to big crunch. In *SIGMOD*.
- [31] Ioanna Tsalouchidou, Gianmarco De Francisci Morales, Francesco Bonchi, and Ricardo Baeza-Yates. 2016. Scalable dynamic graph summarization. In *Big Data*.
- [32] Peixiang Zhao, Charu C Aggarwal, and Min Wang. 2011. gSketch: on query estimation in graph streams. *PVLDB* 5, 3 (2011), 193–204.
- [33] Zhongying Zhao, Chao Li, Xuejian Zhang, Francisco Chiclana, and Enrique Herrera Viedma. 2019. An incremental method to detect communities in dynamic evolving social networks. *Knowledge-Based Systems* 163 (2019), 404–415.
- [34] Yang Zhou, Hong Cheng, and Jeffrey Xu Yu. 2010. Clustering large attributed graphs: An efficient incremental approach. In *ICDM*.



**Figure 6: Effects of the parameters of MoSSo.** (a) MoSSo gave the most compact output when the escape probability  $e$  was around 0.1. As  $e$  increased, MoSSo tended to yield larger outputs without much change in its execution time. (b) As the number of samples  $c$  increased, MoSSo tended to take more time with more compact outputs.



**Figure 7: (a) Effects of input graph properties.** More nodes with similar connectivity (i.e., higher copying probability  $\beta$ ) led to better compression. (b,c) MoSSo is more scalable than MoSSo-SIMPLE. MoSSo took near-constant time per change (i.e., accumulated runtime  $\propto |E|$ ), while MoSSo-SIMPLE took more over time (i.e., accumulated runtime  $\propto |E|^{1.42}$ ).

## A APPENDIX: ADDITIONAL EXPERIMENTS

### A.1 Effects of Parameters (Fig. 6)

We analyzed the effects of the parameters of MoSSo on its speed and compression ratio. To this end, we measured how its accumulated runtime and compression ratio changed depending on the number  $c$  of samples and the escape probability  $e$  in the Skitter dataset.

As seen in Fig. 6, the compression ratio was minimized when the escape probability  $e$  was around 0.1, and the compression ratio tended to increase as  $e$  increased. While the runtime decreased as  $e$  increased, the amount of change was small. As the number of samples  $c$  increased, the compression ratio decreased gradually, while the runtime increased proportionally to  $c$ .

### A.2 Effects of Graph Properties (Fig. 7(a))

To analyze the effect of graph properties on the compression rates of MoSSo, we used the copying model [14] with varying copying probabilities. Specifically, we created synthetic graph with  $|V| = |E| = 10,000$  based on the model<sup>3</sup> and measured compression rates for each copying probability  $\beta$ . As seen in Fig. 7(a), more nodes with similar connectivity (i.e., higher copying probability  $\beta$ ) led to better compression.

### A.3 Additional Scalability Tests (Figs. 7(b)-7(c))

We measured accumulated runtime of MoSSo and MoSSo-SIMPLE in the Web-EU-05 and Skitter datasets. As seen in Figs. 7(b)-7(c), the accumulated runtime of MoSSo was near-linear in both datasets. However, the accumulated runtime of MoSSo-SIMPLE was super-linear (proportional to  $|E|^{1.42}$ ).

## B APPENDIX: PROOFS

**Proof of Thm. 1.** In our solution, we sample twice in a row: we first draw a supernode according to Eq. (2) and then draw a node in the supernode. For any given node  $u \in \bigcup_{i=1}^k S_i$ , the probability of drawing  $u$  is

<sup>3</sup>We symmetrized output graphs and removed all self-loops.

$$P(u \text{ is selected})$$

$$= P(S_u \text{ is selected}) \cdot P(u \text{ is selected} | S_u \text{ is selected})$$

$$= \frac{|S_u|}{|S_1| + \dots + |S_k|} \cdot \frac{1}{|S_u|} = \frac{1}{|S_1| + \dots + |S_k|} = \frac{1}{S},$$

where  $S := |\bigcup_{i=1}^k S_i| = \sum_{i=1}^k |S_i|$ . Hence, each node in  $\bigcup_{i=1}^k S_i$  is equally likely to be drawn under this scheme. Thus, a node not in  $\bigcup_{i=1}^k N_i$  is drawn with probability  $\frac{S-N}{S}$ . In such a case, we fail and repeat the process from beginning. Then, the probability of sampling each node  $w \in \bigcup_{i=1}^k N_i$  is

$$\begin{aligned} P(w \text{ is selected}) &= \sum_{i=1}^{\infty} P(w \text{ is drawn at the } i\text{-th trial}) \\ &= \sum_{i=1}^{\infty} P(w \text{ is drawn at the } i\text{-th trial}) \cdot P(\text{fail } (i-1) \text{ times}) \\ &= \sum_{i=1}^{\infty} \frac{1}{S} \cdot \left(\frac{S-N}{S}\right)^{i-1} = \frac{1}{S} \cdot \frac{1}{1 - \frac{S-N}{S}} = \frac{1}{N}. \end{aligned}$$

□

**Proof of Thm. 2.** To ensure that a sampling scheme converges to a target distribution, the sampling process is set to be ergodic (i.e. any supernode can be proposed regardless of the currently sampled supernode) and to satisfy detailed balance (i.e.  $\pi_i P_{ij} = \pi_j P_{ji}$ ). Hence, for ergodicity, we set the proposal probability distribution in the MCMC method to  $P_{ij} = P(S_j | S_i) = \frac{1}{k}$  for any  $i, j$ . That is, the probability that  $S_j$  is selected as the next supernode when the current supernode is  $S_i$  is  $P_{ij}$ . Even though  $\pi$  and  $P$  do not fulfill detailed balance, from the results in [10], introducing the acceptance probability  $A(S_j, S_i) = \min(1, \frac{\pi_j P_{ji}}{\pi_i P_{ij}}) = \min(1, \frac{|S_j|}{|S_i|})$  with which one accepts each proposed sample guarantees that the sampling scheme asymptotically converges to  $\pi$ . □

**Proof of Lem. 1.** In order to retrieving the neighborhood  $N(u)$  of a node  $u$ , we need to look up all nodes in any supernode in  $N(S_u)$  and those in  $C^+(u)$  and  $C^-(u)$ . Thus, it takes  $O(\sum_{S_v \in N(S_u)} |S_v| +$

**Algorithm 3:** MoSSo-MCMC: a baseline algorithm

---

**Input:** summary graph:  $G_t^*$ , edge corrections:  $C_t$ ,  
input change:  $\{src, dst\}^+ \text{ or } \{src, dst\}^-$   
**Output:** summary graph:  $G_{t+1}^*$ , edge corrections:  $C_{t+1}$

```

1 foreach  $u$  in  $\{src, dst\}$  do
2    $TP(u) \leftarrow N(u), TN(u) \leftarrow N(u)$ 
3   foreach  $y$  in  $TN(u)$  do
4      $CP(y) \leftarrow V_t$ 
5     draw a uniformly random node  $x$  from  $N(y)$ 
6     draw a candidate  $z \in CP(y)$  using Eq. (4)
7     temporarily move  $y$  into  $S_z$ 
8     compute  $\Delta\phi$  (i.e., change in  $\phi$ ) for the proposal
9     compute the acceptance probability  $p$  using Eq. (5)
10    sample  $X \sim \text{uniform}(0, 1)$ 
11    if  $X \leq p$  then
12      accept the proposal and update  $G_t^*, C_t$ 
13       $G_{t+1}^*, C_{t+1} \leftarrow G_t^*, C_t$ 
14 return  $G_{t+1}^*, C_{t+1}$ 

```

---

$|C^-(u)| + |C^+(u)|$ ). Since  $\sum_{v \in N(S_u)} |S_v| + |C^+(u)| = \deg(u) + |C^-(u)|$ , the time complexity becomes  $O(\deg(u) + 2|C^-(u)|)$ . Let  $a_u = |C^-(u)|$  and  $b_u = \deg(u)$ . Then, the average-case time complexity of retrieving the neighborhood of a node is:

$$\frac{\sum_{u \in V} (2a_u + b_u)}{|V|} = \frac{2 \sum_{u \in V} a_u + \sum_{u \in V} b_u}{|V|} = \frac{4|C^-| + 2|E|}{|V|},$$

where the second equality comes from  $\sum_{u \in V} |C^-(u)| = 2|C^-|$  and  $\sum_{u \in V} \deg(u) = 2|E|$ . By the optimal encoding (Sect. 3.1),  $|P| + |C^+| + |C^-| \leq |E|$  holds and thus,  $|C^-| \leq |E|$ . Using this inequality, the average-case time complexity is bounded by  $3 \cdot \deg$ .  $\square$

**Proof of Thm. 3.** A node is drawn from  $C^+(u)$  with probability  $\frac{|C^+(u)|}{\deg(u)}$  and from  $\bigcup_{i=1}^k S_i$  with probability  $\frac{\deg(u) - |C^+(u)|}{\deg(u)}$ . The former takes  $O(1)$ , but the latter requires repeated sampling until a node in  $\bigcup_{i=1}^k S_i$  is drawn. As in the proof of Thm. 1, a neighbor of  $u$  in  $\bigcup_{i=1}^k N_i$  is drawn with probability  $\frac{N}{S}$  in each trial. Thus, the expected number of trials is simply  $\frac{S}{N}$ , and the average-case time complexity of GETRANDOMNEIGHBOR for sampling a node is

$$\begin{aligned} & \frac{|C^+(u)|}{\deg(u)} + \frac{\deg(u) - |C^+(u)|}{\deg(u)} \cdot \frac{S}{N} \\ &= \frac{|C^+(u)|}{\deg(u)} + \frac{\deg(u) - |C^+(u)|}{\deg(u)} \cdot \frac{\deg(u) - |C^+(u)| + |C^-(u)|}{\deg(u) - |C^+(u)|} \\ &= \frac{|C^+(u)|}{\deg(u)} + \frac{\deg(u) - |C^+(u)| + |C^-(u)|}{\deg(u)} = 1 + \frac{|C^-(u)|}{\deg(u)}. \end{aligned}$$

Hence, the average-case time complexity of GETRANDOMNEIGHBOR for  $c$  samples is  $O(c \cdot (1 + \frac{|C^-(u)|}{\deg(u)}))$ .  $\square$

**Proof of Lem. 2.**

$$E[X] = \sum_{u \in V} \left( \frac{a_u}{b_u} \frac{b_u}{\sum_{v \in V} b_v} \right) = \sum_{u \in V} \frac{a_u}{\sum_{v \in V} b_v} = \frac{\sum_{u \in V} a_u}{\sum_{u \in V} b_u}.$$

As in the proof of Lemma 1, we can show  $\frac{\sum_{u \in V} a_u}{\sum_{u \in V} b_u} = \frac{2|C^-|}{2|E|} \leq 1$  and thus  $E[X] \leq 1$ .  $\square$

**Proof of Cor. 1.** Due to the assumption, a node  $u$  is used as the input of GETRANDOMNEIGHBOR with probability proportional to  $\deg(u)$ . By Lemma 2,  $\frac{|C^-(u)|}{\deg(u)} \leq 1$  in average case. Therefore, combined with Thm. 3, the average-case time complexity of GETRANDOMNEIGHBOR for  $c$  samples in Alg. 1 becomes  $O(c)$ .  $\square$

**Proof of Thm. 4.** Alg. 1 maintains  $G^*$  and  $C$  for the current snapshot, and their size is  $O(|V| + |P| + |C^+| + |C^-|)$  in total. Moreover, additional  $O(|V|)$  space is required for storing coarse cluster memberships of nodes. Additionally, to rapidly estimate the saving in  $\phi$ , our implementation maintains the counts of edges between pairs of supernodes, and the number of nonzero counts is upper bounded by  $O(|P| + |C^+|)$ . Hence, the total space complexity is  $O(|V| + |P| + |C^+| + |C^-|)$ .  $\square$

**C APPENDIX: DETAILS OF MOSSO-MCMC**

In MoSSo-MCMC, described in Alg. 3, we adapt the proposal probability distribution and the acceptance probability for Stochastic Block Modeling [23]. Both are described in detail below.

For each testing node  $y$  and its uniformly random neighbor  $x$ , the probability that one among the nodes in each supernode  $S_z \in S$  is proposed is defined as

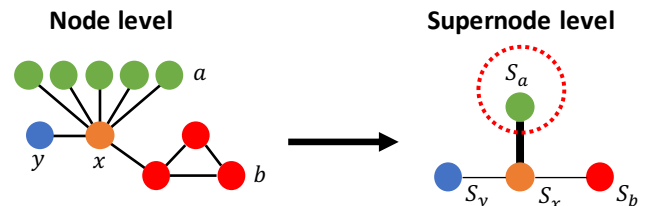
$$p(S_y \rightarrow S_z | S_x) := \frac{|E_{S_z, S_x}| + \epsilon}{|E_{S_x}| + \epsilon |S|}, \quad (4)$$

where  $E_{S_x}$  is the set of edges adjacent to a node in  $S_x$ , and  $\epsilon$  is a parameter that makes every supernode is selected with non-zero probability (i.e. making the process ergodic). As illustrated in Fig. 8, Eq. (4) places importance on supernodes densely connected by many edges to  $S_x$ .

The move of  $y$  into a sampled supernode  $S_z$  is accepted with the probability defined in the Metropolis-Hastings fashion [10] as

$$\min \left( 1, e^{-\beta \Delta\phi} \frac{\sum_{S_x \in S} p_{S_x}^y \cdot p(S_z \rightarrow S_y | S_x)}{\sum_{S_x \in S} p_{S_x}^y \cdot p(S_y \rightarrow S_z | S_x)} \right), \quad (5)$$

where  $p_{S_x}^y := |N(y) \cap S_x| / |N(y)|$ , and  $\beta$  is a parameter used to control how much randomness is injected in accepting the proposed change. The higher  $\beta$  is, the more likely the algorithm is to accept the change even if the change increases  $\phi$ . Note that  $p(S_z \rightarrow S_y | S_x)$  in Eq. (5) should be computed after  $y$  is moved from  $S_y$  to  $S_z$ .



**Figure 8: Proposal probability distribution in MoSSo-MCMC.** (Left) An input graph with the color of nodes indicating their membership to supernodes. (Right) Its summary graph where the thickness of each superedge indicates the number of edges between the supernodes. According to Eq. (4), if  $y$  is a testing node, it is most likely that one among the nodes in  $S_a$  is selected as a candidate.